

Лабораторна робота №4

Тема: Бібліотеки NumPy і Matplotlib у мові Python.

Мета: Набуття практичних навичок застосування бібліотек NumPy і Matplotlib у мові Python.

Лабораторні завдання

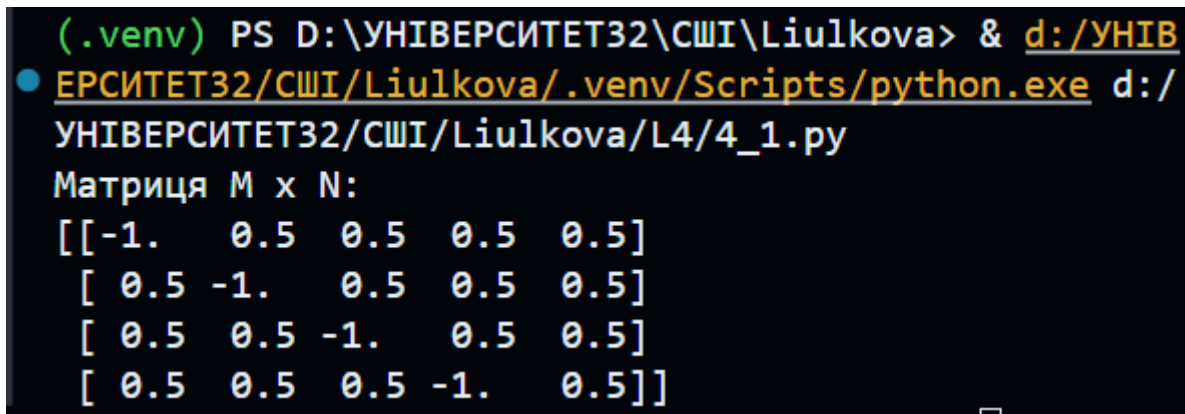
Завдання 1. Створіть матрицю розміру $M \times N$, у якій всі елементи головної діагоналі рівні -1, а всі інші – 0.5. Наприклад, якщо $M=4$, $N=5$, то повинен вийти такий результат:

```
array([[ -1. ,  0.5,  0.5,  0.5,  0.5],
       [ 0.5, -1. ,  0.5,  0.5,  0.5],
       [ 0.5,  0.5, -1. ,  0.5,  0.5],
       [ 0.5,  0.5,  0.5, -1. ,  0.5]])
```

Опис коду розробленої програми

Програма створює масив заданого розміру $M \times N$, заповнюючи його значенням 0.5 за допомогою `np.full`. Далі функція `np.fill_diagonal` змінює елементи головної діагоналі на -1. Результат формування структури показано на рис. 1.1.

Приклади роботи програми:



```
(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/.venv/Scripts/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L4/4_1.py
Матриця M x N:
[[-1.  0.5  0.5  0.5  0.5]
 [ 0.5 -1.  0.5  0.5  0.5]
 [ 0.5  0.5 -1.  0.5  0.5]
 [ 0.5  0.5  0.5 -1.  0.5]]
```

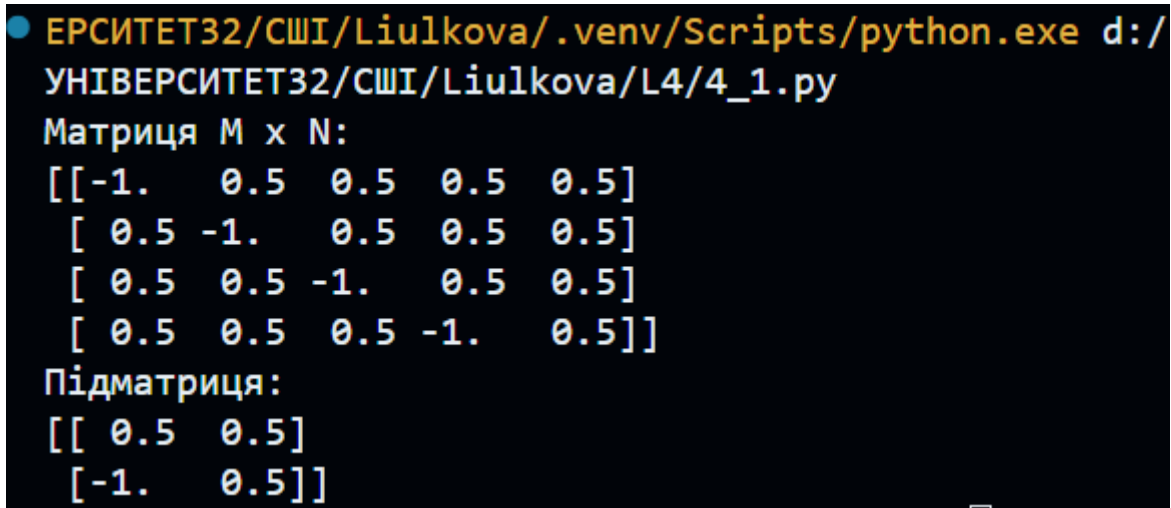
Рис. 1.1 Результат створення матриці з модифікованою головною діагоналлю

Завдання 2. Отримати підматрицю матриці попереднього завдання на основі заданих індексів лівого верхнього і правого нижнього кута цієї матриці. Наприклад, якщо задані індекси `[[0, 1], [1, 2]]`, то повинна вийти наступна підматриця:

```
[[0.5 0.5
 -1. 0.5]]
```

Опис коду розробленої програми

Програма виконує операцію зрізу (slicing) над матрицею з попереднього завдання. Використовуючи передані координати кутів, ми виділяємо прямокутну область даних. Процес виокремлення підматриці продемонстровано на рис. 2.1.

Приклад роботи програми:


```

● EPCITET32/CШI/Liulkova/.venv/Scripts/python.exe d:/
УНІВЕРСИТЕТ32/CШI/Liulkova/L4/4_1.py
Матриця М x N:
[[-1.  0.5  0.5  0.5  0.5]
 [ 0.5 -1.  0.5  0.5  0.5]
 [ 0.5  0.5 -1.  0.5  0.5]
 [ 0.5  0.5  0.5 -1.  0.5]]
Підматриця:
[[ 0.5  0.5]
 [-1.  0.5]]

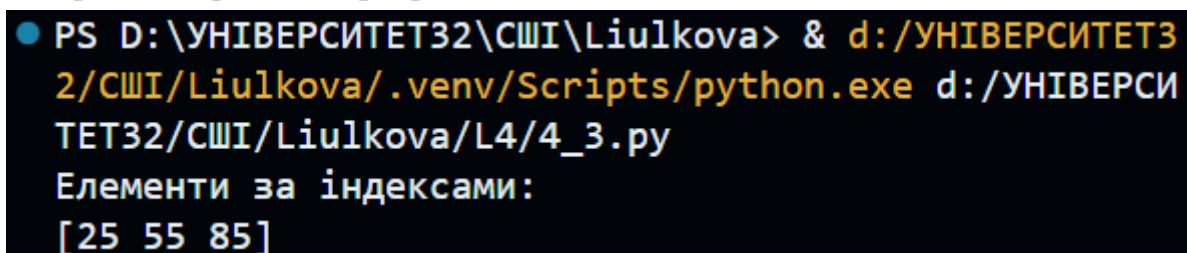
```

Рис. 2.1 Вигляд отриманої підматриці за заданими індексами

Завдання 3. З першого заданого одновимірного масиву отримати масив всіх тих його елементів, індекси яких містяться в другому заданому масиві.

Опис коду розробленої програми

У програмі реалізовано механізм "fancy indexing". З першого масиву вибираються лише ті значення, порядкові номери яких вказані у другому масиві. Логіку вибору елементів за масивом індексів відображено на рис. 3.1.

Приклади роботи програми:


```

● PS D:\УНІВЕРСИТЕТ32\CШI\Liulkova> & d:/УНІВЕРСИТЕТ32/CШI/Liulkova/.venv/Scripts/python.exe d:/УНІВЕРСИТЕТ32/CШI/Liulkova/L4/4_3.py
Елементи за індексами:
[25 55 85]

```

Рис. 3.1 Отриманий масив значень після фільтрації за індексами

Завдання 4. Отримати масив всіх тих елементів заданого одновимірного масиву, які більші (менші) заданого значення.

Опис коду розробленої програми

Програма використовує булеву маску для фільтрації одновимірного масиву. Умова $data > threshold$ створює масив істинності, за яким відбираються потрібні значення. Результат фільтрації значень за порогом наведено на рис. 4.1.

Приклади роботи програми:

```
(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/.venv/Scripts/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L4/4_4.py
Елементи більші за 10:
[12 20 15]
```

Рис. 4.1 Масив елементів, що задовольняють задану нерівність

Завдання 5. Створити матрицю $K \times K$, у якій елементи побічної діагоналі рівні номеру Вашого варіанту N , а решта – 0.

Опис коду розробленої програми

Програма створює квадратну матрицю розміром $K \times K$, що складається з нулів. Оскільки в NumPy немає вбудованої функції для побічної діагоналі, ми використовуємо хитрий прийом: `np.fliplr` віддзеркалює матрицю зліва направо, після чого `np.fill_diagonal` заповнює її звичайну головну діагональ числом 13. У результаті маємо побічну діагональ у початковій матриці. Вигляд матриці наведено на рис. 5.1.

Приклади роботи програми:

```
(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/.venv/Scripts/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L4/4_5.py
Матриця KxK (побічна діагональ):
[[ 0  0  0  0 13]
 [ 0  0  0 13  0]
 [ 0  0 13  0  0]
 [ 0 13  0  0  0]
 [13  0  0  0  0]]
```

Рис. 5.1 Матриця з заповненою побічною діагоналлю згідно з 13 варіантом

Завдання 6. Заповнити масив 7×7 значеннями так, щоб на зазначених на рисунку червоним кольором місцях був номер Вашого варіанту N , а на інших – 0:

0	0	0	N	0	0	0
0	0	N	0	N	0	0
0	N	0	0	0	N	0
N	0	0	0	0	0	N
0	N	0	0	0	N	0
0	0	N	0	N	0	0
0	0	0	N	0	0	0

Опис коду розробленої програми

Для формування фігури "ромб", зображеної в завданні, використовується логічна маска. Функція `pr.indices` генерує індекси всіх елементів. Ми розраховуємо манхеттенську відстань від центрального елемента (3, 3): якщо сума модулів різниць індексів дорівнює 3, клітинка належить до "червоної" зони. Ці клітинки мають значення 13. Структуру отриманого ромба показано на рис. 6.1.

Приклади роботи програми:

```
(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/.venv/Scripts/python.exe d:/
● УНІВЕРСИТЕТ32/СШІ/Liulkova/L4/4_6.py

Матриця:
[[ 0  0  0 13  0  0  0]
 [ 0  0 13  0 13  0  0]
 [ 0 13  0  0  0 13  0]
 [13  0  0  0  0  0 13]
 [ 0 13  0  0  0 13  0]
 [ 0  0 13  0 13  0  0]
 [ 0  0  0 13  0  0  0]]
```

Рис. 6.1

Завдання 7. Створити матрицю розміром $K \times K$, що складається з нулів. Без використання жодного циклу заповнити її номером Вашого варіанту N у шаховому порядку (так, щоб клітинка $[0, 0]$ була нулем, $[0, 1]$ – N і т. д.).

Опис коду розробленої програми

Програма ініціалізує нульову матрицю $K \times K$. Для створення шахового порядку без циклів застосовуються зрізи (slicing) з кроком 2. Спочатку числом

13 заповнюються непарні стовпці в парних рядках [0::2, 1::2], а потім — парні стовпці в непарних рядках [1::2, 0::2]. Завдяки цьому клітинка [0, 0] залишається нулем. Результат заповнення продемонстровано на рис. 7.1.

Приклади роботи програми:

```
(.venv) PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/.venv/Scripts/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L4/4_7.py

Матриця 8x8 (шаховий порядок):
[[ 0 13  0 13  0 13  0 13]
 [13  0 13  0 13  0 13  0]
 [ 0 13  0 13  0 13  0 13]
 [13  0 13  0 13  0 13  0]
 [ 0 13  0 13  0 13  0 13]
 [13  0 13  0 13  0 13  0]
 [ 0 13  0 13  0 13  0 13]
 [13  0 13  0 13  0 13  0]]
```

Рис. 7.1 Масив $K \times K$, заповнений числом 13 у шаховому порядку

Завдання 8. Згенерувати одновимірний масив із K випадкових чисел у діапазоні від L до N . Нормалізувати його так, щоб всі значення опинилися в

діапазоні від 0 до 1 за формулою: $x^{\boxed{?}} = \frac{x - x_{min}}{x_{max} - x_{min}}$. Побудувати

стовпчасту діаграму (plt.bar) для початкового та нормалізованого масивів (на двох різних графіках поруч).

Опис коду розробленої програми

Програма використовує np.random.randint для генерації одновимірного масиву розміром K . Далі за допомогою вбудованих функцій min() та max() знаходяться крайні значення для застосування заданої математичної формули нормалізації (використовуються векторні операції масивів). Для візуалізації matplotlib створює два графіки поруч за допомогою plt.subplots(1, 2), де функція ax.bar буде відповідні стовпчасті діаграми. Візуалізацію результатів наведено на рис. 8.1.

Приклади роботи програми:

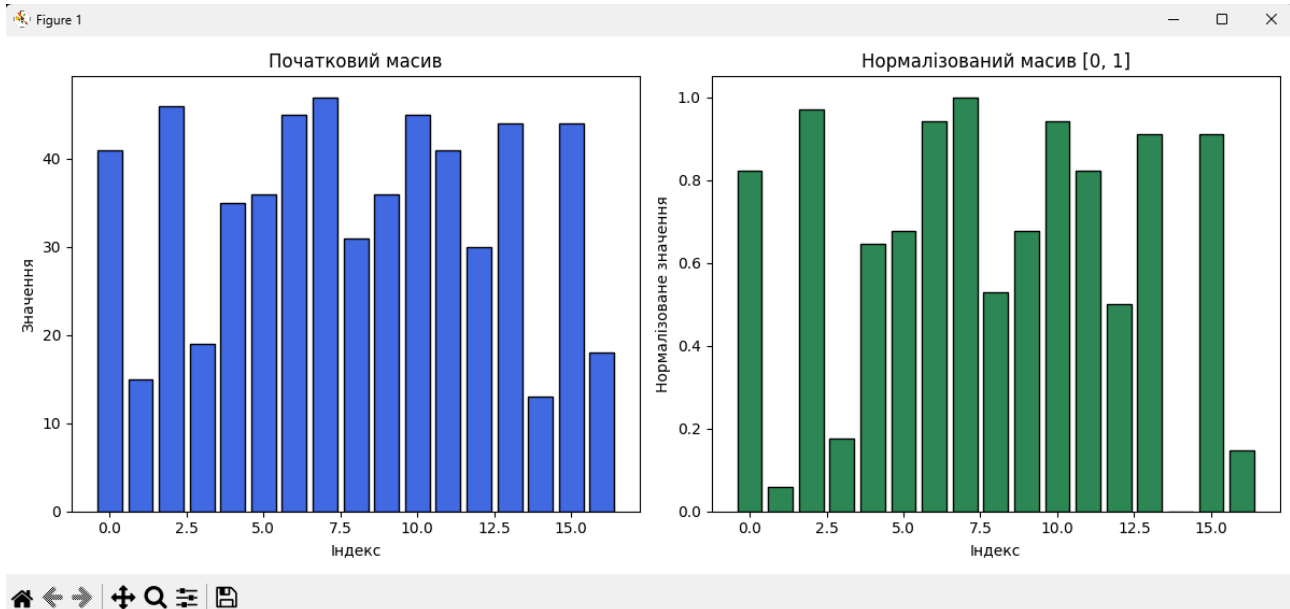


Рис. 8.1 Столпчасті діаграми початкового (зліва) та нормалізованого (справа) масивів випадкових чисел

Завдання 9. Створити двовимірний масив (матрицю) розміру $K \times L$ з випадкових цілих чисел від 0 до 255 (імітація зображення у відтінках сірого з шумом). Застосувати до нього операцію згладжування: кожен внутрішній елемент (не крайній) повинен бути замінений на середнє арифметичне його самого та його 8 сусідів. У програмі не можна використовувати цикли.

Опис коду розробленої програми

Для виконання операції без циклів використано потужний механізм слайсингу (зрізів) бібліотеки NumPy. Кожна група індексів [a:b, c:d] виділяє відповідну підматрицю сусідів (зверху-зліва, зверху-по центру і т.д.). Їх сума ділиться на 9, і результат масово присвоюється внутрішнім елементам матриці [1:-1, 1:-1]. Для демонстрації роботи як імітації зображення додатково виведено графіки через plt.imshow (див. рис. 9.1).

Приклади роботи програми:

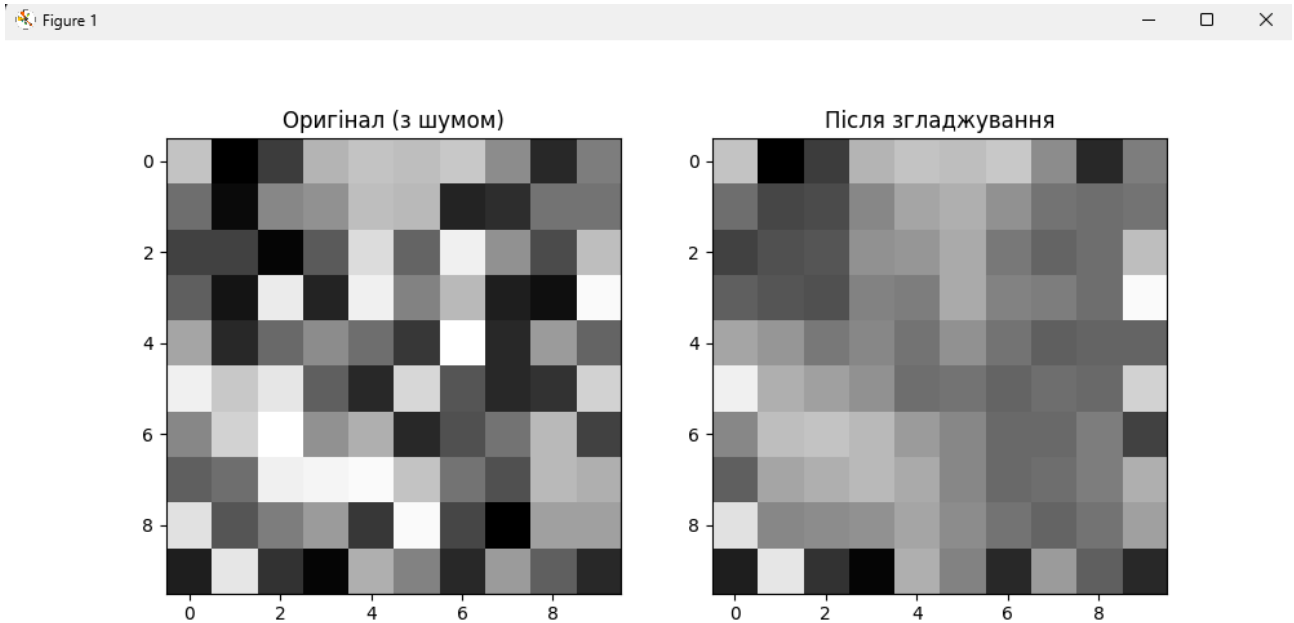


Рис. 9.1 Візуалізація двовимірного масиву як зображення до та після векторного згладжування матриці

Завдання 10. Побудувати графік складної функції свого варіанту на заданому проміжку з завдання 1 файлу L1_Py_t.pdf.

$$13) \frac{n-1}{\cos x} + \frac{n-2}{\sin 2x} - \frac{n-3}{\cos 3x} + \frac{n-4}{\sin 4x} + \frac{n-5}{\cos 5x} - \frac{n-6}{\sin 6x} + \dots$$

Опис коду розробленої програми

Програма адаптує вашу попередню логіку обчислення членів ряду для роботи з векторами NumPy. За допомогою `np.linspace` генерується масив із 1000 значень x . Цикл проходить по доданках ряду (від 1 до n), знаходячи знак, чисельник і знаменник (`np.cos` або `np.sin`), та додає їх до підсумкового масиву y . Для коректного відображення графіка (оскільки тригонометричні функції в знаменнику створюють розриви), значення y обмежуються за допомогою `np.clip`, після чого будується графік за допомогою `plt.plot`. Результат наведено на рис. 10.1. та рис. 10.2.

Приклади роботи програми:

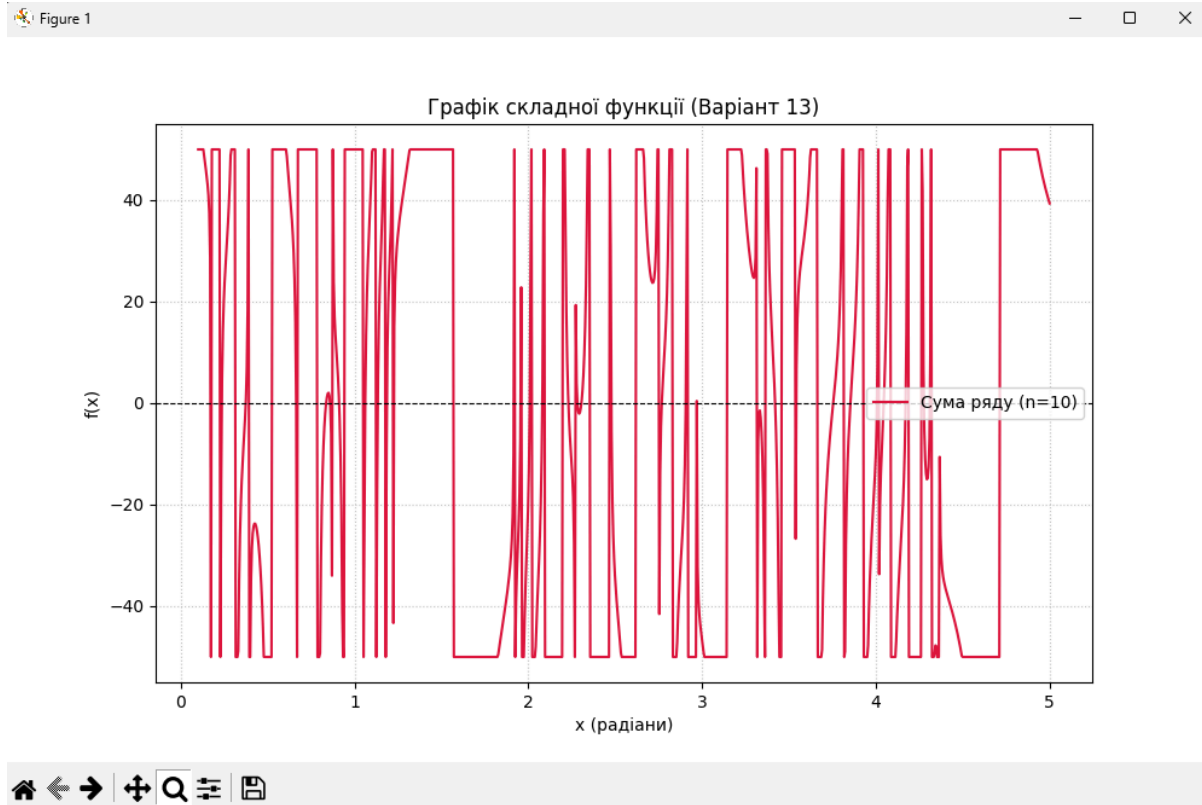


Рис. 10.1. Графік значень складної функції для 13 варіанту на проміжку $[0.1, 5]$. Масштаб від -50 до 50 за віссю y

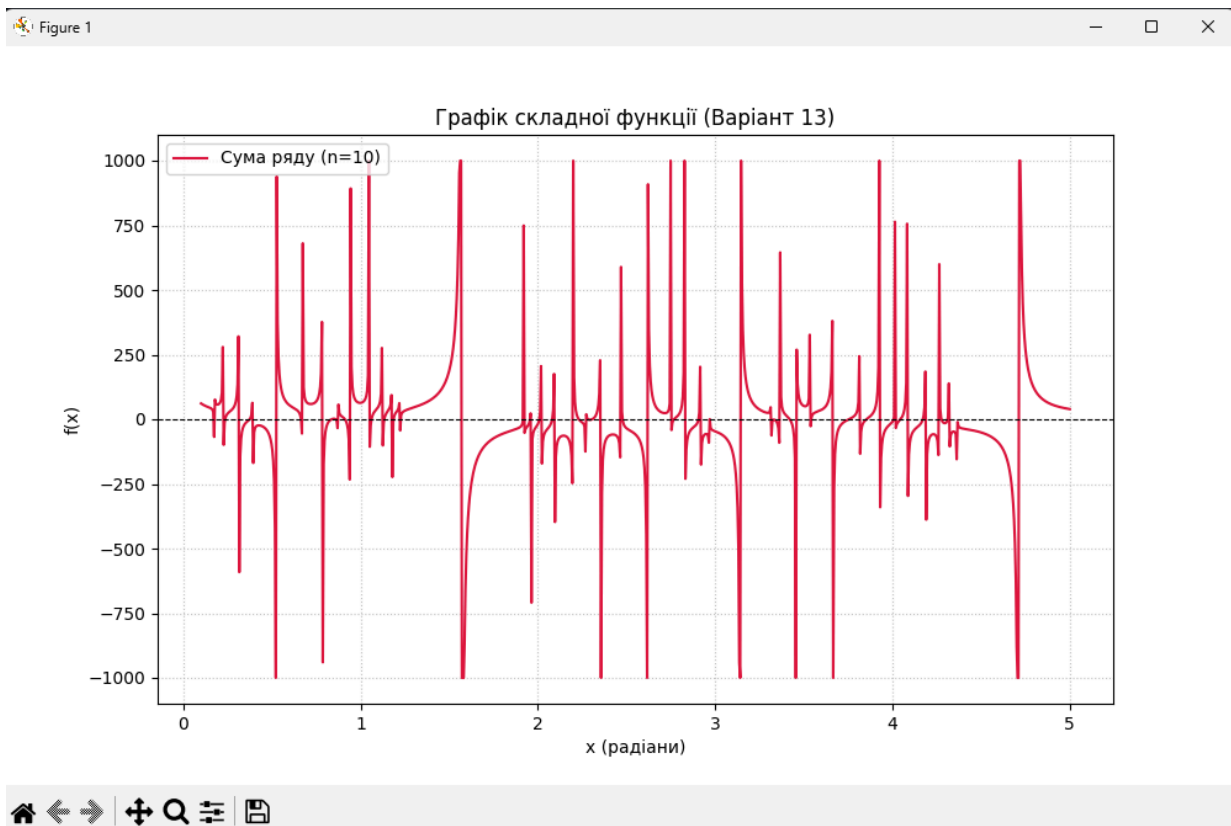


Рис. 10.2. Графік значень складної функції для 13 варіанту на проміжку $[0.1, 5]$. Масштаб від -1000 до 1000 за віссю y

Висновки. Під час виконання лабораторної роботи було успішно набуто практичних навичок роботи з бібліотеками NumPy та Matplotlib у мові Python. Було засвоєно методи створення та ініціалізації масивів різних розмірностей, застосування масок та "fancy indexing" для фільтрації даних.

Особливу увагу приділено принципам "векторизації" в NumPy, що дозволяє виконувати складні масові обчислення та операції з матрицями (зокрема шахове заповнення та згладжування двовимірного масиву) без використання класичних циклів, що суттєво підвищує продуктивність коду. Також було застосовано інструментарій Matplotlib для наочної візуалізації даних: побудови лінійних графіків складних математичних функцій з точками розриву, стовпчастих діаграм для порівняння розподілів та імітації відтворення зашумлених зображень у відтінках сірого.